Lecture 06: Practice Problems & Assignment 5 — Master Solutions



🌐 **Language / भाषा: English** | हिंदी / Hinglish | ⬅️ Back to Questions Suite

---

1. Overview & Pedagogical Objective

This module contains the rigorous, modular, production-standard implementations for all problems in **Assignment 5** (Python Fundamentals Part 5) and advanced enterprise systems tasks.

---

2. Master Implementation & Test Harness

```
import os
import json
from datetime import datetime

# ========================================
# Solution 1: Names Registry (Assignment 5 Q1)
# ========================================
def execute_assignment_q1_names(filepath: str = "names.txt", names_list: list = No
    """
    Opens file in write mode, writes 5 names (one per line),
    then re-opens in read mode and returns all lines.
    """
    if names_list is None:
        names_list = ["Aarav", "Diya", "Kabir", "Ananya", "Rohan"]

    # Step 1 & 2: Open in write mode and write names
    with open(filepath, "w", encoding="utf-8") as f_out:
        for name in names_list:
            f_out.write(name + "\n")

    # Step 3: Open in read mode and read names
    read_names = []
    with open(filepath, "r", encoding="utf-8") as f_in:
        for line in f_in:
            read_names.append(line.strip())

    return read_names

# ========================================
# Solution 2: Audit Logging (Assignment 5 Q2)
# ========================================
def execute_assignment_q2_logging(filepath: str = "log.txt", new_entry: str = "Pro
    """
    Opens file in append mode, adds a timestamped log entry,
    then opens in read mode and returns all historical logs.
    """
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    formatted_entry = f"[{timestamp}] {new_entry}\n"

    # Step 1 & 2: Append mode
    with open(filepath, "a", encoding="utf-8") as f_log:
        f_log.write(formatted_entry)

    # Step 3: Read and print all logs
    all_logs = []
    with open(filepath, "r", encoding="utf-8") as f_read:
        all_logs = [line.strip() for line in f_read.readlines()]

    return all_logs
```

```
# ========================================
# Solution 3: List Comprehension (Assignment 5 Q3)
# ========================================
def execute_assignment_q3_comprehension(input_numbers: list = None) -> list:
    """
    Uses list comprehension to filter only numbers strictly greater than 15.
    """
    if input_numbers is None:
        input_numbers = [5, 10, 15, 20, 25]

    filtered = [x for x in input_numbers if x > 15]
    return filtered

# ========================================
# Solution 4: City JSON Store (Assignment 5 Q4)
# ========================================
def execute_assignment_q4_json_workflow(filepath: str = "cities.json", new_city: s
    """
    Saves initial 3 cities to cities.json, loads it, prints cities,
    updates with new city & population, and persists updated JSON.
    """
    initial_cities = {
        "Delhi": 33000000,
        "Mumbai": 21000000,
        "Bengaluru": 13500000
    }

    # Step 1: Save initial dictionary to cities.json
    with open(filepath, "w", encoding="utf-8") as f_out:
        json.dump(initial_cities, f_out, indent=4)

    # Step 2: Load JSON and read
    with open(filepath, "r", encoding="utf-8") as f_in:
        loaded_cities = json.load(f_in)

    # Step 3: Add new city and population, update JSON file
    loaded_cities[new_city] = new_pop
    with open(filepath, "w", encoding="utf-8") as f_update:
        json.dump(loaded_cities, f_update, indent=4)

    # Step 4: Verify final file
    with open(filepath, "r", encoding="utf-8") as f_final:
        final_dict = json.load(f_final)

    return final_dict

# ========================================
# Solution 5: Missing File Guard (Assignment 5 Q5)
# ========================================
```

```
def execute_assignment_q5_file_guard(filepath: str = "data.txt") -> str:
    """
    Attempts to open data.txt in read mode. If file does not exist,
    catches FileNotFoundError and returns 'File not found!'.
    """
    try:
        with open(filepath, "r", encoding="utf-8") as f:
            content = f.read()
            return f"File content read: {content}"
    except FileNotFoundError:
        return "File not found!"

# ========================================
# Solution 6: Enterprise Server Log Incident Analyzer (Systems Engineering)
# ========================================
def execute_enterprise_log_analyzer(log_filepath: str, report_filepath: str) -> di
    """
    Parses web server access logs, extracts HTTP status codes, filters errors
    using list comprehension, and exports structured audit_report.json.
    """
    with open(log_filepath, "r", encoding="utf-8") as f:
        log_lines = f.readlines()

    # Extract status codes using list comprehension
    # Format expected: IP - - [TIMESTAMP] "METHOD PATH HTTP/1.1" STATUS BYTES
    records = []
    for line in log_lines:
        parts = line.strip().split()
        if len(parts) >= 9:
            status_code = int(parts[8])
            records.append({"endpoint": parts[6], "status": status_code})

    total_requests = len(records)
    error_records = [r for r in records if r["status"] >= 400]
    success_records = [r for r in records if r["status"] < 400]

    error_rate = (len(error_records) / total_requests * 100) if total_requests > 0

    report = {
        "generated_at": datetime.now().isoformat(),
        "total_requests": total_requests,
        "successful_requests": len(success_records),
        "failed_requests": len(error_records),
        "error_rate_percent": round(error_rate, 2),
        "incident_alerts": [r["endpoint"] for r in error_records if r["status"] ==
    }

    with open(report_filepath, "w", encoding="utf-8") as f_out:
        json.dump(report, f_out, indent=4)
```

```
    return report

# Driver execution suite
print("=== Assignment 5 Master Solutions Execution ===")

# Q1 Execution
q1_res = execute_assignment_q1_names("names.txt")
print(f"\nQ1 (Names Registry Readback): {q1_res}")

# Q2 Execution
q2_res = execute_assignment_q2_logging("log.txt", "Assignment 5 Automated Verifica
print(f"\nQ2 (Audit Logs Count): {len(q2_res)} lines logged.")

# Q3 Execution
q3_res = execute_assignment_q3_comprehension([5, 10, 15, 20, 25])
print(f"\nQ3 (Filtered > 15): {q3_res}")

# Q4 Execution
q4_res = execute_assignment_q4_json_workflow("cities.json", "Hyderabad", 10500000)
print(f"\nQ4 (Updated Cities JSON): {list(q4_res.keys())}")

# Q5 Execution
q5_res = execute_assignment_q5_file_guard("non_existent_data.txt")
print(f"\nQ5 (Missing File Exception Message): '{q5_res}'")

# Q6 Execution
sample_logs = """192.168.1.10 - - [20/Oct/2026:10:00:00] "GET /api/v1/users HTTP/1
192.168.1.11 - - [20/Oct/2026:10:00:05] "POST /api/v1/checkout HTTP/1.1" 500 512
192.168.1.12 - - [20/Oct/2026:10:00:10] "GET /assets/style.css HTTP/1.1" 200 4096
192.168.1.13 - - [20/Oct/2026:10:00:15] "GET /unknown-page HTTP/1.1" 404 256"""

with open("server_access.log", "w", encoding="utf-8") as f:
    f.write(sample_logs)
q6_res = execute_enterprise_log_analyzer("server_access.log", "audit_report.json")
print(f"\nQ6 (Systems Log Analyzer Error Rate): {q6_res['error_rate_percent']}%")

# Cleanup created test files
for temp_f in ["names.txt", "log.txt", "cities.json", "server_access.log", "audit_
    if os.path.exists(temp_f):
        os.remove(temp_f)
```

---

3. Mathematical & Algorithmic Analysis

Problem 3 List Comprehension Big-O Complexity

Filtering sequence $X = (x_1, \dots, x_N)$ under predicate $P(x) = (x > 15)$:

$$\text{Time Complexity: } \mathcal{O}(N), \qquad \text{Space Complexity: } \mathcal{O}(K), \quad K = \sum_{i=1}^{N} \mathbb{I}(x_i > 15) \leq$$

Problem 6 Error Rate Formulation

Let access logs contain requests $\mathcal{R} = \{r_1, \ldots, r_M\}$ with HTTP response code $C(r_i)$:

$$\text{Error Rate } (\%) = \frac{\sum_{i=1}^{M} \mathbb{I}(C(r_i) \geq 400)}{M} \times 100\%$$

---

4. Key Takeaways & Best Practices

- **Idiomatic Resource Management**: Always pair file I/O with context managers (`with open`) to ensure deterministic closure.
- **Defensive Parsing**: Handle JSON IO and file lookups inside structured `try...except` boundaries.
- **Vectorized Comprehensions**: Prefer list comprehensions over accumulator loops for linear filtering and mapping.